

Kaunas University of Technology
Faculty of Informatics

Machine Learning Algorithms

Laboratory work No. 2

Decision Trees and Random Forest

Student: Vladyslava Yaroshenko
Group: IFGU-4
Dataset: Quebec Housing Sales (target variable: sale_price)

Kaunas, 2026

Introduction

This laboratory work implements a regression tree, a classification tree and a random forest from scratch and compares them with the scikit-learn versions. The data is the Quebec Housing Sales set from Laboratory work No. 1 (5,000 sales, target sale_price).

- split the data into training, validation and test samples and fill the missing values;
- implement and tune a regression tree and compare it with DecisionTreeRegressor;
- turn the price into five classes, implement a classification tree and compare it with DecisionTreeClassifier;
- implement a random forest, tune it with 5-fold cross-validation, test it and predict a single new record.

1. Data and Preparation

The identifier property_id was dropped, which leaves 12 features and the target sale_price (mean 480,214, median 453,000 dollars). Unlike in Laboratory work 1 no dummy variables are needed, because the implemented trees split on text categories directly.

Table 1. Features used in the models

Feature	Type	Remark
city	categorical (32)	multiway split
neighborhood	categorical (72)	multiway split
property_type	categorical (6)	Single Family, Condo, Townhouse, Duplex, Triplex, Bungalow
garage, basement	categorical (Yes/No)	two branches
bedrooms, bathrooms	numerical (counts)	five thresholds from distinct values
living_area_sqft, lot_size_sqft	numerical	lot size is missing for condos
year_built, renovation_year, sale_year	numerical (years)	renovation year is missing if never renovated
sale_price	numerical, target	in classification converted to 5 classes

The data was split randomly into 3,500 training, 750 validation and 750 test records (70/15/15 %, random_state = 42). Missing values (lot_size_sqft 24.2 %, renovation_year 68.2 %) are filled with training medians only, and no scaling is needed because a tree only compares values with thresholds.

2. Regression Decision Tree

2.1 Implementation

In a regression tree the leaf value is the mean of the target, and the impurity of a node S is the MSE around that mean:

$$\mathrm{MSE}(S) = \frac{1}{|S|} \sum_{i \in S} (y_i - \bar{y}_S)^2 \tag{1}$$

$$\bar{y}_S = \frac{1}{|S|} \sum_{i \in S} y_i \tag{2}$$

A candidate split divides S into K parts, and the split with the smallest size-weighted impurity is chosen, which is the largest variance reduction:

$$Q = \sum_{k=1}^{K} \frac{|S_k|}{|S|} \cdot \mathrm{MSE}(S_k) \tag{3}$$

A categorical feature gets one branch per category (a binary feature gives two branches). A numerical feature is tested at five thresholds, the 0.1, 0.3, 0.5, 0.7 and 0.9 quantiles of its distinct values in the node, and x ≤ t goes left:

$$t_j = Q_{q_j}(u), \qquad q_j \in \{0.1; 0.3; 0.5; 0.7; 0.9\} \tag{4}$$

Growth stops at max_depth, when a node has fewer than min_samples_split or 2·min_samples_leaf records, when its MSE is zero or when no split improves the criterion; a split with a branch smaller than min_samples_leaf is rejected. A category that was not seen in a node returns the mean price of that node.

```
class RegressionTree:
    def __init__(self, max_depth=5, min_samples_split=10,
                 min_samples_leaf=5):
        self.max_depth = max_depth
        self.min_samples_split = min_samples_split
        self.min_samples_leaf = min_samples_leaf

    def mse(self, y):
        return np.mean((y - np.mean(y)) ** 2)

    def best_split(self, X, y):
        best_error = self.mse(y)
        best = None

        for feature in X.columns:
            values = X[feature]

            if pd.api.types.is_numeric_dtype(values):
                unique = np.sort(values.unique())

                if len(unique) <= 1:
                    continue

                if len(unique) == 2:
                    thresholds = [unique[0]]
                else:
                    thresholds = np.quantile(
                        unique, np.linspace(0.1, 0.9, 5)
                    )

                for threshold in np.unique(thresholds):
                    left = values <= threshold
                    right = ~left

                    if min(left.sum(), right.sum()) < self.min_samples_leaf:
                        continue

                    error = (
                        left.mean() * self.mse(y[left]) +
```

```
                    right.mean() * self.mse(y[right])
                )

                if error < best_error:
                    best_error = error
                    best = (feature, "numeric", threshold)

        else:
            categories = values.unique()

            if len(categories) < 2:
                continue

            groups = [values == c for c in categories]

            if any(g.sum() < self.min_samples_leaf for g in groups):
                continue

            error = sum(
                g.mean() * self.mse(y[g])
                for g in groups
            )

            if error < best_error:
                best_error = error
                best = (feature, "categorical", categories)

    return best

def build(self, X, y, depth=0):
    node = {"prediction": float(y.mean())}

    if (
        depth >= self.max_depth or
        len(y) < self.min_samples_split or
        len(y) < 2 * self.min_samples_leaf or
        self.mse(y) == 0
    ):
        return node

    split = self.best_split(X, y)

    if split is None:
        return node

    feature, kind, value = split

    node["feature"] = feature
    node["kind"] = kind

    if kind == "numeric":
```

```
            node["threshold"] = float(value)

            mask = X[feature] <= value

            node["left"] = self.build(
                X[mask], y[mask], depth + 1
            )
            node["right"] = self.build(
                X[~mask], y[~mask], depth + 1
            )

        else:
            node["branches"] = {}

            for category in value:
                mask = X[feature] == category
                node["branches"][category] = self.build(
                    X[mask], y[mask], depth + 1
                )

        return node

    def fit(self, X, y):
        self.tree = self.build(X, y)
        return self

    def predict_one(self, row, node):
        if "feature" not in node:
            return node["prediction"]

        feature = node["feature"]

        if node["kind"] == "numeric":
            if row[feature] <= node["threshold"]:
                return self.predict_one(row, node["left"])
            return self.predict_one(row, node["right"])

        branch = node["branches"].get(row[feature])

        if branch is None:
            return node["prediction"]

        return self.predict_one(row, branch)

    def predict(self, X):
        return np.array([
            self.predict_one(row, self.tree)
            for _, row in X.iterrows()
        ])
```

2.2 Five parameter combinations

The quality is measured with the MAE, the MSE (and its root) and the MAPE:

$$\mathrm{MAE} = \frac{1}{n}\sum_{i=1}^{n} |y_i - \hat{y}_i| \tag{5}$$

$$\mathrm{MSE} = \frac{1}{n}\sum_{i=1}^{n} (y_i - \hat{y}_i)^2, \qquad \mathrm{RMSE} = \sqrt{\mathrm{MSE}} \tag{6}$$

$$\mathrm{MAPE} = \frac{100\%}{n}\sum_{i=1}^{n} \frac{|y_i - \hat{y}_i|}{|y_i|} \tag{7}$$

Five combinations of (max depth, min split, min leaf) were trained on the training sample and judged on the validation sample; the results are in Table 2.

Table 2. Own regression tree: training and validation errors (MAE in dollars, MSE in 10^9)

Depth	Min split	Min leaf	Train MAE	Val. MAE	Train MSE	Val. MSE	Train MAPE, %	Val. MAPE, %
3	10	5	80,328	81,497	10.98	10.63	18.76	19.00
5	10	5	69,157	78,102	8.58	10.32	16.35	18.31
7	10	5	62,339	80,540	7.08	11.29	14.75	18.80
5	20	10	94,555	99,184	14.24	15.05	22.45	23.34
7	20	10	89,483	101,641	13.05	15.79	21.18	23.98

The combination (5, 10, 5) is the best on all validation metrics (MAE 78,102, MAPE 18.3 %). Depth 3 underfits and depth 7 starts to overfit (training MAE 62.3k against validation 80.5k), and the two combinations with min leaf 10 are clearly worse, probably because a multiway split on 32 cities or 72 neighbourhoods rarely keeps every branch above the leaf limit (not tested separately).

2.3 Comparison with scikit-learn

The same five combinations were run with DecisionTreeRegressor on the one-hot encoded data (Table 3).

Table 3. DecisionTreeRegressor: training and validation errors (MAE in dollars, MSE in 10^9)

Depth	Min split	Min leaf	Train MAE	Val. MAE	Train MSE	Val. MSE	Train MAPE, %	Val. MAPE, %
3	10	5	103,038	104,265	16.92	16.69	24.54	24.49
5	10	5	90,242	93,853	13.63	14.01	21.60	22.20
7	10	5	77,630	84,709	10.65	11.89	18.58	20.01
5	20	10	90,370	94,777	13.63	14.06	21.72	22.37
7	20	10	79,631	86,525	11.14	12.46	19.16	20.34

The best scikit-learn tree (7, 10, 5) has a validation MAE of 84,709, so my best tree (78,102) is about 7.8 % better, but for min leaf 10 the library is better (94.8k and 86.5k against 99.2k and 101.6k). My tree uses a whole category in one level, whereas one-hot encoding spreads the location over more than a hundred columns, while the library checks all numerical thresholds and mine only five.

2.4 First three layers of the best tree

The root of the best tree (5, 10, 5) splits by property type, then by city for single-family houses and by sale year for the other types.

This is reasonable: the type and the location decide the price level, and then the living area or the year of sale refine it.

3. Classification Decision Tree

3.1 Mapping of the price to five classes

The price is converted into five classes with the 20th, 40th, 60th and 80th percentiles of the training prices:

$$\tau_k = Q_{0.2k}(y_{\text{train}}), \qquad k = 1,2,3,4 \tag{8}$$

The limits are 333,000; 412,600; 499,000 and 617,200 dollars, and each class has about 700 training records, so a random guess is right in 20 % of cases.

Table 4. Price classes

Class	Rule	Training records
Very Low	Price < 333,000	698
Low	333,000 ≤ Price < 412,600	702
Medium	412,600 ≤ Price < 499,000	696
High	499,000 ≤ Price < 617,200	704
Very High	Price ≥ 617,200	700

3.2 Implementation

ClassificationTree inherits from RegressionTree and replaces the impurity with the Gini index and the leaf value with the most frequent class:

$$G(S) = 1 - \sum_{c=1}^{5} p_c^2, \qquad p_c = \frac{n_c}{|S|} \tag{9}$$

$$\hat{c}(S) = \arg\max_c \, p_c \tag{10}$$

Everything else (multiway categorical splits, five numerical thresholds and the stopping rules) is the same as in section 2.1.

```
class ClassificationTree(RegressionTree):

    def gini(self, y):
        probabilities = np.bincount(
            np.asarray(y, dtype=int), minlength=5
        ) / len(y)

        return 1 - np.sum(probabilities ** 2)

    def mse(self, y):
        return self.gini(y)

    def build(self, X, y, depth=0):
```

```
        counts = np.bincount(
            np.asarray(y, dtype=int), minlength=5
        )
        node = {"prediction": int(np.argmax(counts))}

        if (
            depth >= self.max_depth or
            len(y) < self.min_samples_split or
            len(y) < 2 * self.min_samples_leaf or
            self.gini(y) == 0
        ):
            return node

        split = self.best_split(X, y)

        if split is None:
            return node

        feature, kind, value = split

        node["feature"] = feature
        node["kind"] = kind

        if kind == "numeric":
            node["threshold"] = float(value)
            mask = X[feature] <= value

            node["left"] = self.build(
                X[mask], y[mask], depth + 1
            )
            node["right"] = self.build(
                X[~mask], y[~mask], depth + 1
            )

        else:
            node["branches"] = {}

            for category in value:
                mask = X[feature] == category

                node["branches"][category] = self.build(
                    X[mask], y[mask], depth + 1
                )

        return node
```

3.3 Metrics and five parameter combinations

The metrics are calculated from the confusion matrix; precision, recall and specificity are macro-averaged over the five classes (TP, FP, FN and TN are counted class against the rest):

$$\text{Accuracy} = \frac{\sum_{c=1}^{5} TP_c}{n} \tag{11}$$

$$\text{Precision}_c = \frac{TP_c}{TP_c + FP_c} \tag{12}$$

$$\text{Recall}_c = \frac{TP_c}{TP_c + FN_c} \tag{13}$$

$$\text{Specificity}_c = \frac{TN_c}{TN_c + FP_c} \tag{14}$$

Table 5. Own classification tree: metrics on the training and validation samples

Depth	Min split	Min leaf	Training				Validation			
			Acc.	Prec.	Recall	Spec.	Acc.	Prec.	Recall	Spec.
3	10	5	0.6297	0.6345	0.6297	0.9074	0.4920	0.4942	0.4894	0.8734
5	10	5	0.7031	0.7036	0.7032	0.9258	0.5027	0.4953	0.4984	0.8758
7	10	5	0.7123	0.7129	0.7123	0.9281	0.5160	0.5102	0.5123	0.8792
5	20	10	0.6017	0.6026	0.6017	0.9004	0.4893	0.4826	0.4859	0.8725
7	20	10	0.6023	0.6026	0.6023	0.9006	0.4907	0.4835	0.4872	0.8729

The combination (7, 10, 5) has the highest validation accuracy (0.516), which is more than twice the 20 % of a random guess. The training accuracy is 0.712, so the tree overfits, and the difference between depths 5 and 7 (1.3 points) is within the noise of about 1.8 points that 750 records give.

3.4 Confusion matrices

Figure 1 shows the confusion matrices of the selected tree on the training and validation samples.

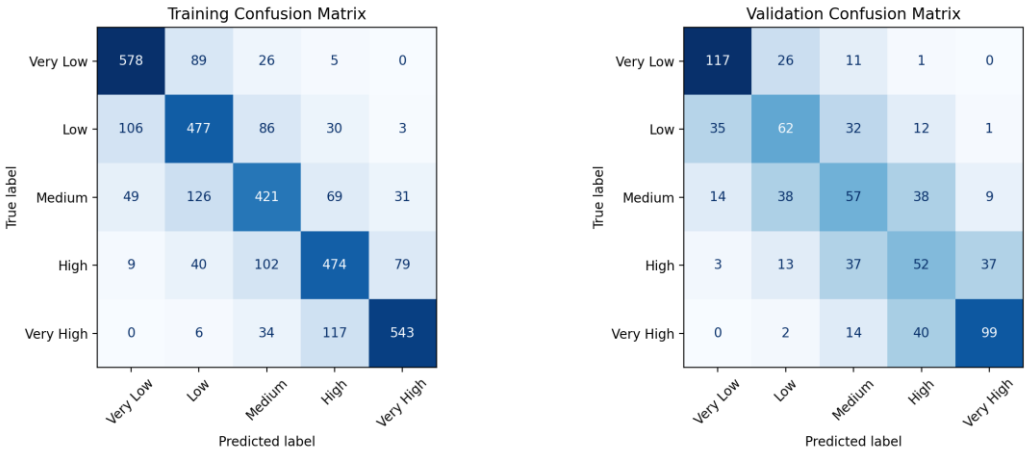


Figure 1. Confusion matrices of the selected tree (7, 10, 5): training (left) and validation (right) samples

On the validation sample 387 of 750 records (51.6 %) are classified correctly, and 89.3 % are correct or wrong by one class only. The recall is best for the cheapest (75.5 %) and the most expensive (63.9 %) houses and lowest for the middle classes (36.5–43.7 %).

3.5 Comparison with scikit-learn

The same combinations were run with DecisionTreeClassifier (Table 6, Figure 2).

Table 6. DecisionTreeClassifier: metrics on the training and validation samples

Depth	Min split	Min leaf	Training				Validation			
			Acc.	Prec.	Recall	Spec.	Acc.	Prec.	Recall	Spec.
3	10	5	0.4083	0.4363	0.4078	0.8520	0.3800	0.3456	0.3817	0.8458
5	10	5	0.4854	0.4886	0.4854	0.8714	0.4653	0.4470	0.4601	0.8662
7	10	5	0.5446	0.5371	0.5447	0.8861	0.5013	0.4814	0.4963	0.8753
5	20	10	0.4854	0.4906	0.4854	0.8714	0.4627	0.4409	0.4573	0.8655
7	20	10	0.5389	0.5339	0.5389	0.8847	0.5200	0.5047	0.5161	0.8801

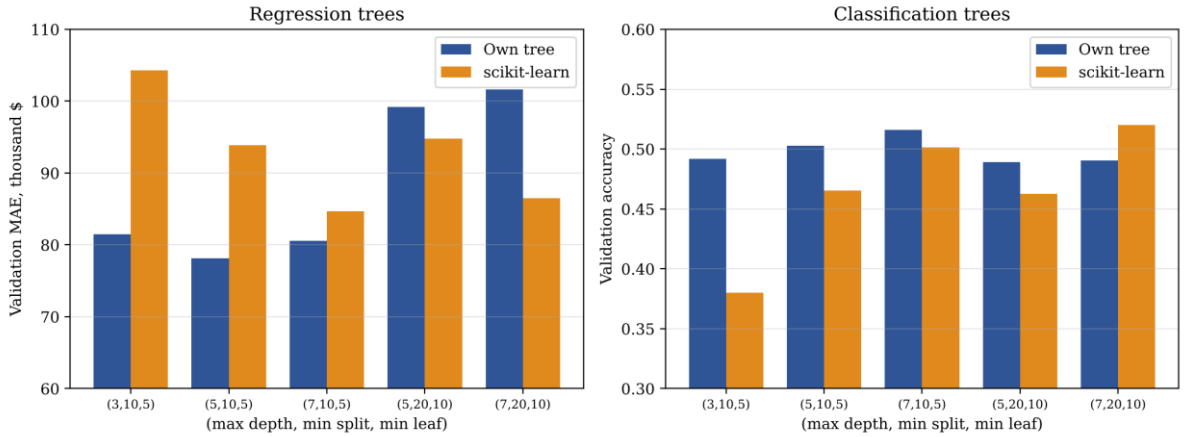


Figure 2. Own trees and scikit-learn trees on the validation sample

The best scikit-learn tree (7, 20, 10) has a validation accuracy of 0.520 against 0.516 for mine, so the two are equally good. My tree overfits more (training accuracy 0.712 against about 0.54), but it is clearly better for shallow trees (0.492 against 0.380 at depth 3) because of the multiway split.

3.6 First three layers of the best tree

The root of the selected tree (7, 10, 5) is a 72-way split by neighborhood, which the Gini criterion prefers to any numerical feature.

4. Random Forest for Regression

4.1 Implementation

A single deep tree has a high variance, so a forest averages B trees that are made different by randomness. Each tree is trained on a bootstrap sample (n records drawn with replacement), and a record is left out with the probability

$$P(\text{not chosen}) = \left(1 - \frac{1}{n}\right)^n \to e^{-1} \approx 0.368 \tag{15}$$

so one tree sees about 63 % of the distinct records. Each tree also gets a random subset of m = ⌊ √p⌋ features, here 3 of 12:

$$m = \lfloor \sqrt{p} \rfloor = \lfloor \sqrt{12} \rfloor = 3 \tag{16}$$

The forest prediction is the average of the trees, formula (17). If the trees have variance σ² and the average correlation ρ, the variance of this average is given by formula (18).

$$\hat{y}(x) = \frac{1}{B}\sum_{b=1}^{B} T_b(x) \tag{17}$$

$$\mathrm{Var}(\hat{y}) = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2 \tag{18}$$

More trees therefore help only up to a point, and the trees should be as uncorrelated as possible. In my implementation the feature subset is chosen once per tree and not at every split as in Breiman's forest, which is a simplification that affects the results.

```
class RandomForest:
    def __init__(self, n_trees=10, max_depth=5,
                 min_samples_leaf=5, random_state=42):

        self.n_trees = n_trees
        self.max_depth = max_depth
        self.min_samples_leaf = min_samples_leaf
        self.random_state = random_state
        self.trees = []

    def fit(self, X, y):
        self.trees = []
        rng = np.random.default_rng(self.random_state)

        n_samples = len(X)
        n_features = max(1, int(np.sqrt(X.shape[1])))

        for i in range(self.n_trees):

            # Bootstrap sample
            indices = rng.choice(
                n_samples, size=n_samples, replace=True
            )

            # Random subset of features
            features = rng.choice(
                X.columns, size=n_features, replace=False
            ).tolist()

            X_sample = X.iloc[indices][features].reset_index(drop=True)
            y_sample = y.iloc[indices].reset_index(drop=True)

            tree = RegressionTree(
                max_depth=self.max_depth,
                min_samples_split=2 * self.min_samples_leaf,
                min_samples_leaf=self.min_samples_leaf
            )

            tree.fit(X_sample, y_sample)
            self.trees.append((tree, features))

        return self
```

```
    def predict(self, X):
        predictions = []

        for tree, features in self.trees:
            predictions.append(tree.predict(X[features]))

        return np.mean(predictions, axis=0)
```

4.2 Cross-validation

The training and validation samples are joined (4,250 records) and 5-fold cross-validation (shuffled, random_state = 42) is run on the full grid of 5/10/15 trees, depth 3/5/7 and min leaf 5/10/20, i.e. 27 combinations. The result of a combination is the mean and the standard deviation of the fold MAE:

$$\mathrm{MAE}_{\mathrm{cv}} = \frac{1}{k}\sum_{f=1}^{k} \mathrm{MAE}_f \tag{19}$$

$$s = \sqrt{\frac{1}{k}\sum_{f=1}^{k} \left(\mathrm{MAE}_f - \mathrm{MAE}_{\mathrm{cv}}\right)^2} \tag{20}$$

Table 7. 5-fold cross-validation of the 27 combinations (MAE in dollars; std over the 5 folds)

Trees	Depth	Min leaf	Own forest, mean MAE	Own, std	scikit-learn, mean MAE	sk-learn, std
5	3	5	97,305	5,055	101,151	2,025
5	3	10	99,940	6,779	101,155	2,020
5	3	20	107,035	1,318	101,398	2,233
5	5	5	95,613	4,841	88,551	2,063
5	5	10	98,264	6,987	89,431	1,966
5	5	20	105,395	1,579	90,574	2,245
5	7	5	95,653	4,721	79,762	2,697
5	7	10	98,006	7,016	81,829	2,516
5	7	20	105,247	1,729	84,516	2,928
10	3	5	90,321	1,782	101,057	1,947
10	3	10	93,354	4,529	101,071	1,926
10	3	20	110,528	1,512	101,299	2,079
10	5	5	88,172	1,692	88,106	1,953
10	5	10	91,516	4,832	88,739	1,994
10	5	20	109,309	1,562	90,040	2,329
10	7	5	88,218	1,839	78,446	2,238
10	7	10	91,376	4,969	80,252	2,075
10	7	20	109,276	1,604	83,409	2,620
15	3	5	91,084	1,486	100,909	1,836
15	3	10	94,146	2,701	100,908	1,832
15	3	20	110,382	2,455	101,137	1,958
15	5	5	88,823	1,362	88,056	1,821
15	5	10	92,216	2,765	88,624	1,778
15	5	20	108,972	2,479	89,872	2,041
15	7	5	88,658	1,519	78,275	2,064
15	7	10	91,990	2,871	79,984	2,162

Trees	Depth	Min leaf	Own forest, mean MAE	Own, std	scikit-learn, mean MAE	sk-learn, std
15	7	20	108,849	2,397	83,327	2,211

4.3 Comparison with scikit-learn and effect of parameters

RandomForestRegressor was evaluated on the same grid and the same folds with one-hot encoded data.

Figure 3 shows the fold MAE for 5, 10 and 15 trees, and Figure 4 shows the effect of the number of trees and the best forest of each kind (Table 8).

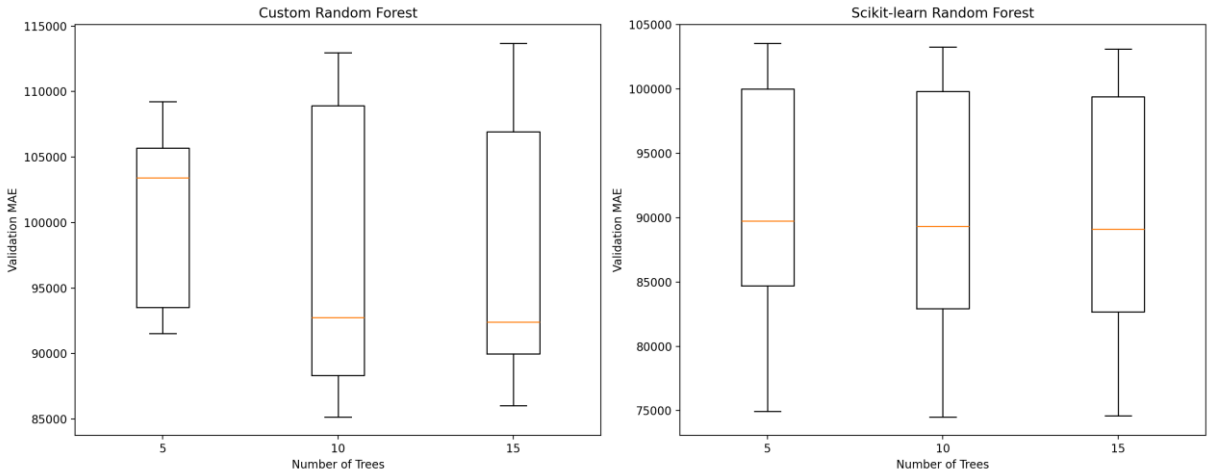


Figure 3. Distribution of the fold MAE for 5, 10 and 15 trees (all depth and leaf combinations, all folds)

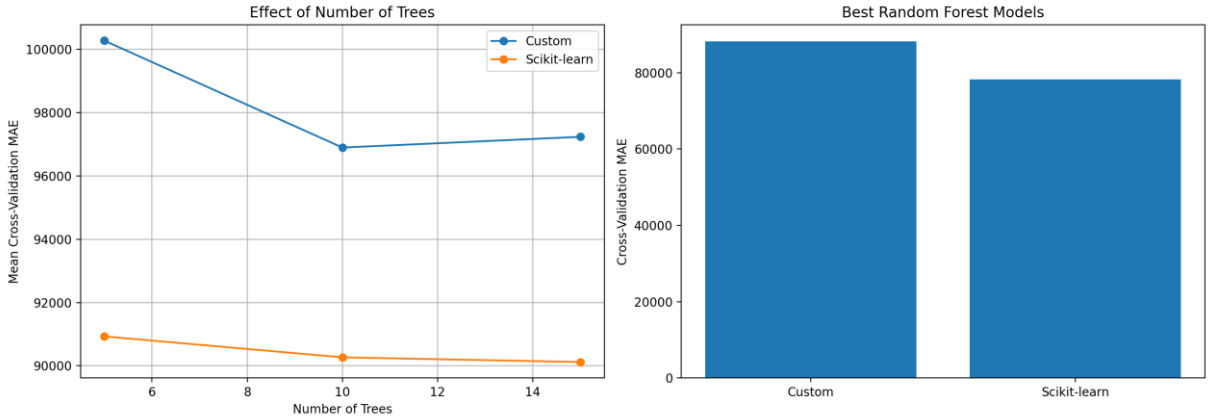


Figure 4. Left: mean cross-validation MAE against the number of trees; right: the best forest of each kind

Table 8. Effect of the number of trees (MAE in dollars)

Trees	Own: best combination	Own: mean of 9	scikit-learn: best	scikit-learn: mean of 9
5	95,613	100,273	79,762	90,930
10	88,172	96,897	78,446	90,269
15	88,658	97,236	78,275	90,121

The best own forest is 10 trees, depth 5, min leaf 5 (MAE 88,172, std 1,692), and the best scikit-learn forest is 15 trees, depth 7, min leaf 5 (78,275), which is about 11 % better. For my forest the step from 5 to 10 trees helps (95,613 to 88,172) and 15 trees bring nothing, in line with the 1/B term in formula (18).

Small leaves are better (mean MAE 91.5k, 94.5k and 108.3k for leaf 5, 10 and 20), and a depth above 5 does not help because each tree sees only 3 of 12 features, so many trees miss the strongest predictors. For the same

reason my forest (88,172) is worse than my single tree (78,102), while the library forest improves on its single tree (78.3k against 84.7k).

4.4 Final model and test results

The best forest (10 trees, depth 5, min leaf 5) was refitted on all 4,250 records and applied once to the 750 test records, with text categories left as they are.

The test MAE is 90,847 and the MSE is $1.3199 \cdot 10^{10}$ (RMSE 114,888 dollars), only about 3 % above the cross-validation MAE, so the choice of parameters did not overfit. The MAE is still about 19 % of the mean price.

Table 9. The first ten test records

No.	Actual price	Predicted price	Absolute error
1	560,000	494,726	65,274
2	354,000	498,724	144,724
3	613,000	554,088	58,912
4	636,000	562,801	73,199
5	229,000	406,701	177,701
6	489,000	449,896	39,104
7	729,000	516,926	212,074
8	715,000	549,630	165,370
9	773,000	655,435	117,565
10	518,000	541,057	23,057

The predictions lie between 407k and 655k while the real prices go from 229k to 773k, so the forest pulls the answers toward the middle.

4.5 Prediction for a single record

The fitted forest is saved as final_forest.pkl, and a new record can be priced without training: missing numeric values are replaced by the training medians and the columns are put in the stored order.

For the townhouse in Table 10 (never renovated, so the renovation year is the median 2016) the predicted price is 440,009 dollars.

Table 10. Example of an input record

Feature	Value	Feature / value
city	Québec	bathrooms: 2
neighborhood	Charlesbourg	living_area_sqft: 1,250
property_type	Townhouse	lot_size_sqft: 3,000
bedrooms	3	year_built: 1995
garage / basement	Yes / Yes	renovation_year: not given
sale_year	2025	predicted price: 440,009 $

A category that the forest has never seen does not cause an error, because the node simply returns its average price.

5. Conclusions

The regression tree (validation MAE 78,102, MAPE 18.3 %) and the classification tree (validation accuracy 51.6 % on five balanced classes) work directly with categorical features and match or beat scikit-learn on validation data. They overfit more, and their advantage at shallow depth comes from the multiway split.

The best own forest (10 trees, depth 5, leaf 5) has a cross-validated MAE of 88,172 and a test MAE of 90,847 dollars, which is worse than the scikit-learn forest (78,275) and than my single tree. The most likely cause is that each tree gets only three random features for its whole life.

First I would choose the random features at every split and then try more and deeper trees. Predicting the logarithm of the price might also reduce the error, which is still about 19 % of the mean price.